

快速入门

1.1 概述

1.1.1 二次开发框架介绍

星际 GIS 统一协同开发框架由 WebGL 场景可视化应用系统（以下简称“可视化系统”或“前端应用系统”）、WebGL 中控管理系统（以下简称“中台”或“中控”）、后台服务管理平台（以下简称“后台”或“后台服务”）三部分组成。涵盖了数据与功能服务注册、用户角色权限管理、场景定制化组装、多维空间信息一体可视化等过程，用户可根据自身业务需求，零/低代码、低门槛快速搭建数字孪生场景，应用于规划、市政、水务、交通、应急等不同行业领域。

(1) WebGL 场景可视化应用系统

“协同平台前端插件化开发包”指的就是供二次开发使用的该系统的前端工程项目。

WebGL 场景可视化应用系统可对二维基础地理信息数据、三维模型、地形、正射影像数据、倾斜摄影数据、点云数据、综合管线数据、BIM 等多源、海量的空间数据进行统一汇聚管理，实现多维时空数据的可视化和应用。前端应用系统已完成 100+应用功能开发，满足规划、国土、地下空间、城市建设等行业需求，用户可以直接使用无需二次开发。



图1 前端可视化

(2) WebGL 中控管理系统

作为配置组装中满足组织机构管理，用户角色管理，数据、服务以及功能三类资源的管理，可提供个性化应用。可实现定制化服务中台，提供场景组装管理，能够为用户配置系统样式、图层场景和应用功能，实现业务系统的配置化搭建。

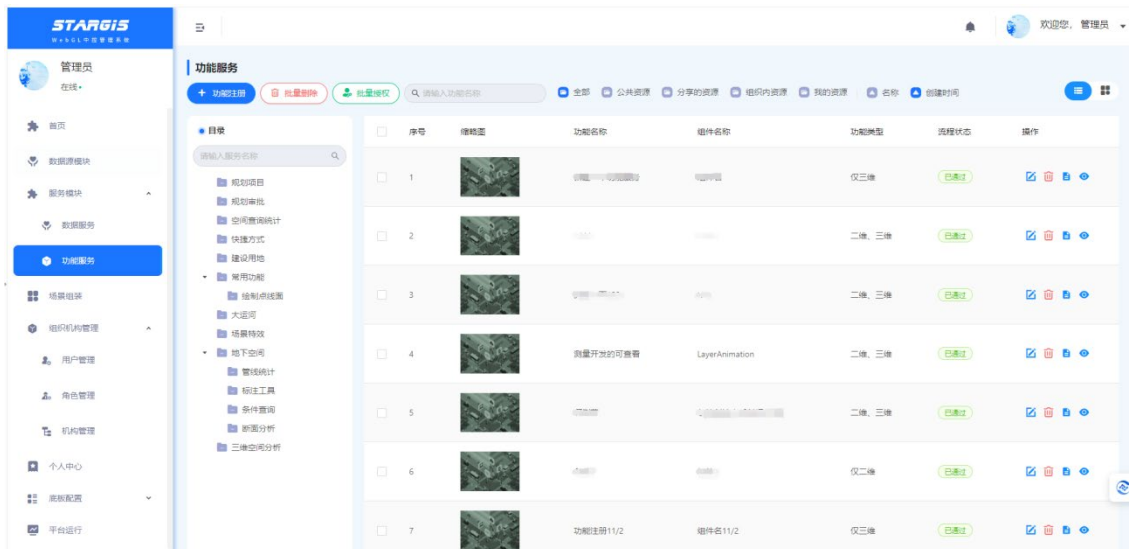


图2 中台服务配置管理

(3) 后台发布数据服务与功能服务

通过 StarGIS EARTH Server 发布复杂的空间分析服务以及基于 StarGIS Earth 平台 JNI 功能服务接口，包括通用的查询、统计、分析以及规划、国土、地下空间业务相关的后台服务。StarGIS EARTH Server 基于 SOA 架构管理各类空间数据服务，可用于系统部署和网络服务发布，兼容第三方的 MapServer、FeatureServer、数据库等多种数据服务。



图3 后台分析服务

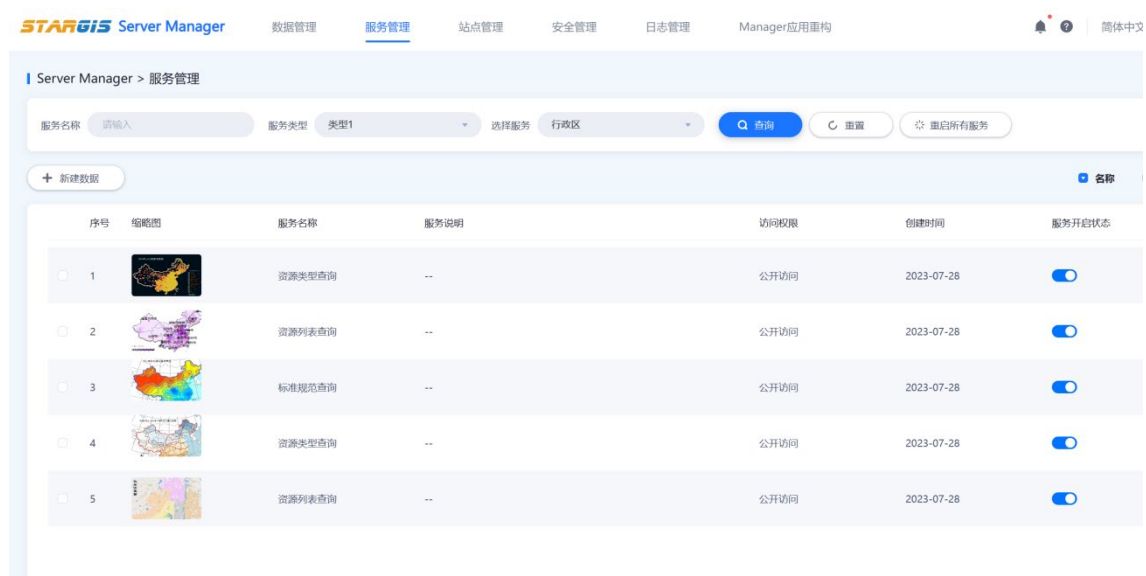


图4 STARGIS EARTH Server 软件

1.1.2 二次开发环境要求

Windows 环境下首先应满足一下基本软硬件环境配置：

配置名称	基本要求	推荐配置
CPU	主频不低于 2GHz	3GHz 以上多核心处理器
内存	不低于 16G	32G 以上
硬盘空间	不低于 10G 可用空间	50G 以上可用空间、固态硬盘
显卡	显存不低于 4G 的独立显卡	NVIDIA RTX 3070 8G 显存或更高级别游戏卡
操作系统	Windows 7/10/11(64 位)	Windows10(64 位)
浏览器	Google 浏览器 64 位版本不低于 83	版本 100 以上

其次需要以下开发环境支持：

开发环境	说明
Node.js	JavaScript 运行环境
Yarn	项目依赖包管理工具
Vue-CLI	Vue.js 工程化脚手架工具
Visual Studio Code	代码开发编辑器
tomcat	Web 应用服务器
Google 浏览器（版本 100 以上）	开发环境调试

1.2 项目环境配置与运行

1.2.1 开发环境搭建

(1) 安装 Visual Studio Code

开发时推荐使用 Visual Studio Code（简称“VS Code”）可从以下地址下载安装：

<https://code.visualstudio.com/>。为了提升开发效率，可在 Visual Studio Code 中安装以下插件：

- ESLint：代码检查工具
- Prettier：代码格式化工具
- Vue (Official)：Vue 代码支持

(2) Node.js 环境安装

项目基于 Node.js 运行前端构建脚本及包管理工具。建议安装长期支持版本（LTS），例如 Node.js 16.x 或 18.x。可通过官方网站下载安装包进行安装：<https://nodejs.org/>

安装完成后，可通过 cmd 命令行验证版本是否成功：

```
npm -v
```

(3) 安装 Yarn

Yarn 是包管理工具，相较于 npm 具有更快的依赖安装速度和更好的依赖锁定机制。注意使用本项目框架时避免使用 npm 新增和管理依赖，不然会造成包管理混乱。安装 yarn 的 cmd 命令：

```
npm install -g yarn
```

验证是否安装成功：

```
yarn -v
```

(4) 安装 Vue CLI

Vue CLI 是 Vue.js 官方提供的脚手架工具，用于快速初始化和管理工作。由于本项目基于 Vue 2 版本，需安装对应版本的 Vue CLI，通过 Yarn 安装 Vue CLI 的 cmd 命令：

```
yarn global add @vue/cli@3.3.0
```

安装完成后可通过以下命令验证版本：

```
vue -V
```

注意执行 `vue -V` 时，得到的是 Vue CLI 的版本，而不是 `vue` 的。因为在基于 Vue CLI 创建的项目中，Vue 的运行时库（如 `vue@2.x`）通常已经在项目的 `package.json` 中定义，会在后面安装项目依赖时安装，因此这里不需要手动全局安装 `vue` 本身。

1.2.2 项目安装与启动

(1) 安装项目依赖

在 VS Code 中打开项目文件夹，点击上方菜单栏【终端】-【新建终端】，执行以下命令安装项目依赖包：

```
yarn install
```

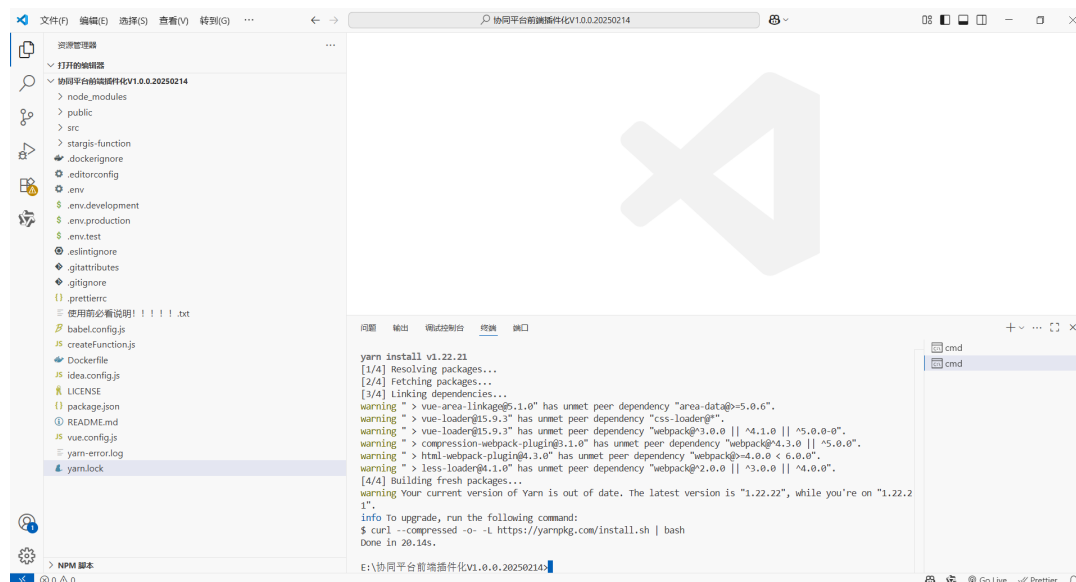


图5 安装项目依赖

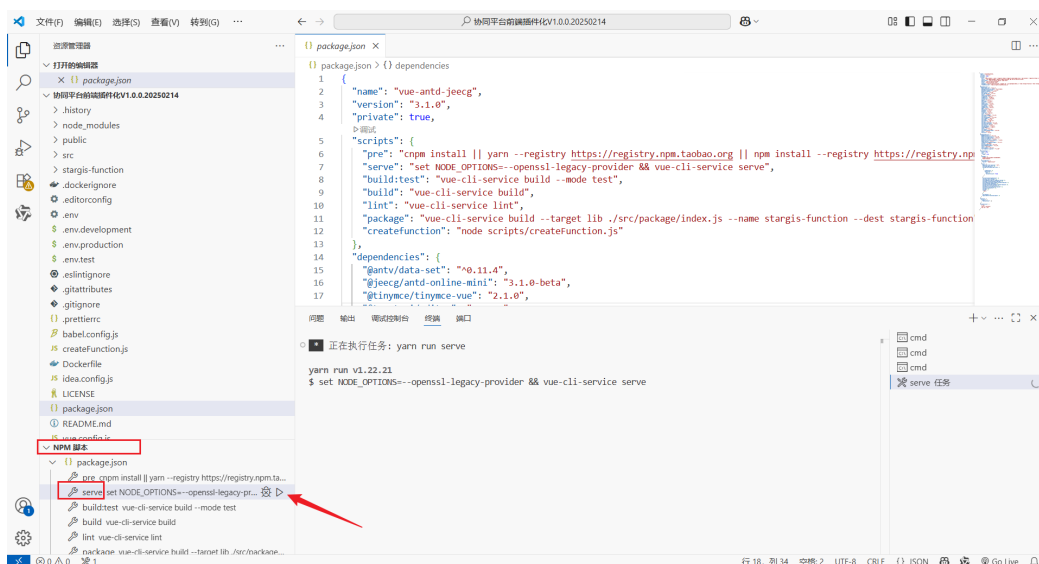
如果使用的是前端插件化开发包，安装项目依赖后还需要将项目根目录的 `stargis-function` 文件夹手动复制到 `node_modules` 文件夹内，再执行后续操作。

(2) 启动开发环境

方式一：在 VS Code 终端执行以下命令：

```
yarn serve
```

方法二：在 VS Code 左侧侧边栏 NPM 脚本中点击 `serve` 指令右侧的运行按钮自动执行：

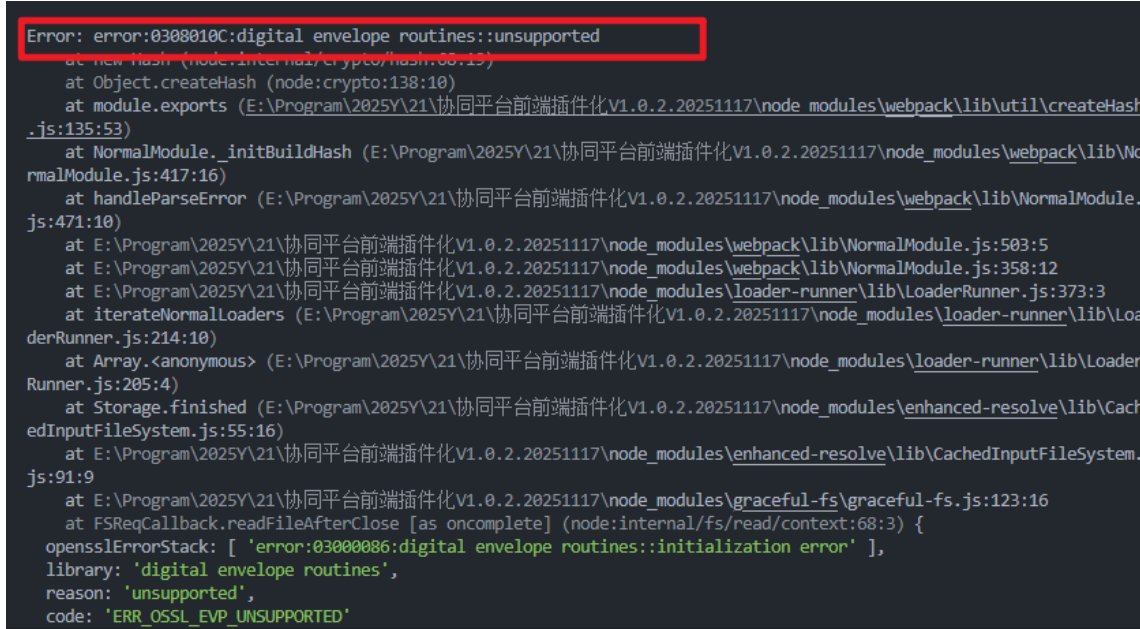


注意：

如果运行环境 Node 版本为 18 以上（版本查询指令为“node -v”），执行上面的 serve 指令（vue-cli-service serve）会报错，将 package.json 中“scripts”对象里 server 属性改为以下内容即可，直接使用左侧侧边栏 NPM 脚本中 **serve_v18** 指令：

```
"serve": "set NODE_OPTIONS=--openssl-legacy-provider && vue-cli-service serve"
```

Node 版本造成的报错示意图：



启动完成后 Vue CLI 会自动启动本地开发服务器（基于 webpack-dev-server），监听端口默认为 3000。终端会输出项目开发环境访问地址，如下图所示。开发环境支持热重载，变更后保存代码浏览器页面将自动刷新，便于调试和开发。

```
问题  输出  调试控制台  终端  端口

st/cjs.js??ref--0-0!./node_modules/vue-loader/lib??vue-loader-options!./src/views/maps/index.vue?vue&type=script&lang=js&
@ ./src/views/maps/index.vue?vue&type=script&lang=js&
@ ./src/views/maps/index.vue
@ ./src/utlis/util.js
@ ./src/permission.js
@ ./src/main.js
@ multi (webpack)-dev-server/client?http://192.168.1.100:3000/sockjs-node (webpack)/hot/dev-server.js ./src/main.js

App running at:
- Local:   http://localhost:3000/
- Network: http://192.168.1.100:3000/

Note that the development build is not optimized.
To create a production build, run yarn build.
```

1.2.3 项目打包与部署

(1) 项目打包

在开发完成后，需将项目构建为静态资源以供部署。执行以下命令进行打包：

```
yarn build
```

构建过程中，Vue CLI 会根据 vue.config.js 中的配置执行构建任务，生成的打包文件位于 dist 目录。构建结果为 stargisWebGL 文件夹包括 index.html 页面及若干静态资源文件（JavaScript、CSS、图像等），已进行压缩优化。

(2) 部署至 Tomcat

构建完成后，将 dist/stargisWebGL 文件夹拷贝至 Tomcat 的 webapps 目录下指定路径。即可通过以下地址访问部署好的前端项目：

```
http://<服务器地址>:<端口>/stargisWebGL
```

附：StarGIS EARTH V4（如 v4.4.1）版本软件自带 tomcat，可将 stargisWebGL 文件夹直接拷贝至..\StargisPlugins-runtime\tomcat\tomcat9\webapps 下，端口为 38080。

1.3 项目结构与开发规范

项目基于 Vue 与 Vue CLI 构建，采用 Ant Design Vue 作为主力 UI 组件库。整体结构遵循标准的 Vue 项目规范，集成了路由管理、状态管理及模块化组件体系，具备良好的可维护性和可扩展性。框架还内置了权限控制机制和 SSO 单点登录功能，满足企业级系统在安全性与用户管理方面的核心需求。

1.3.1 目录结构说明

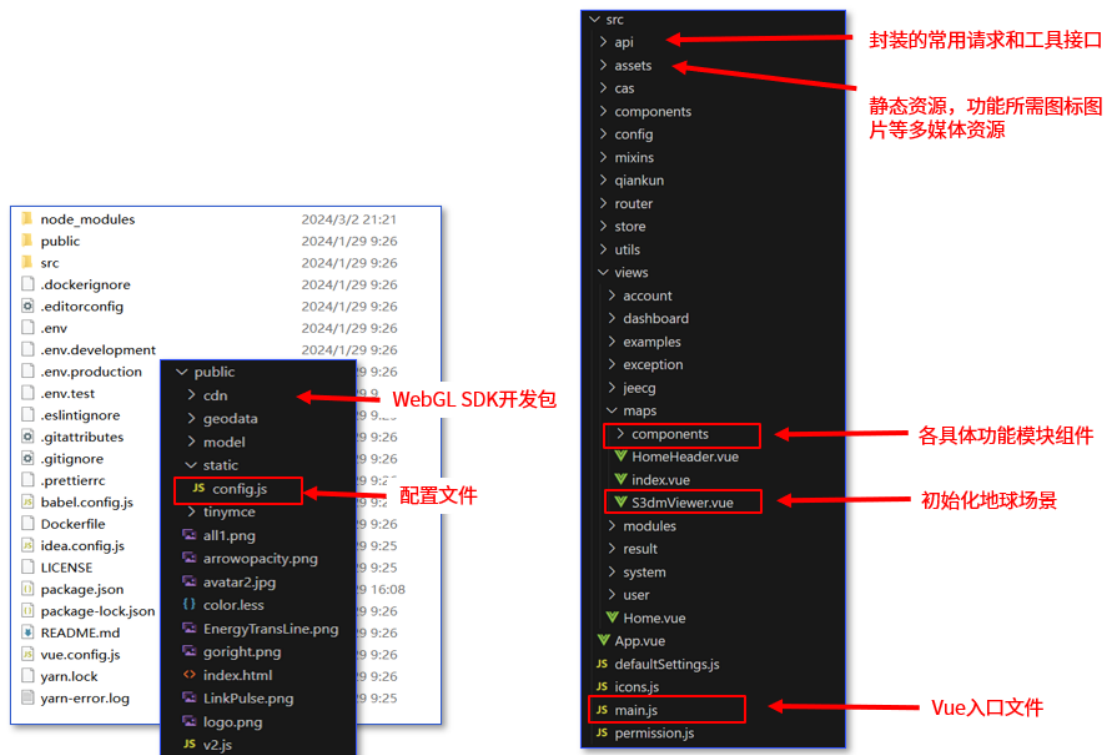


图6 项目结构介绍

二次开发框架项目结构主要包括：

(1) src 目录

项目源代码主目录，包含所有业务逻辑和功能实现。其中主要包括：

1) api 目录

封装所有后端接口请求模块，采用模块化管理方式，将不同业务领域的接口进行分类组织。该目录包含接口定义文件、请求拦截器配置、错误处理机制以及业务接口分组管理。通过统一的 axios 封装，实现了请求的统一处理、响应拦截、错误重试等功能，确保接口调用的稳定性和一致性。

2) assets 目录

静态资源文件存储中心，采用功能模块化的组织方式。该目录包含项目所需的图片资源、样式文件、字体图标、地图按钮图标等静态资源。其中 map-btns 子目录按功能模块细分为多个子目录，每个子目录包含对应功能模块的图标资源，便于资源的统一管理和维护。

3) views 目录

页面视图组件，按业务模块组织，包含各功能页面的 Vue 组件实现。该目录采用模块化的组织方式，每个模块包含对应的页面组件和业务逻辑。

4) components 目录

可复用组件库，是项目组件化开发的核心目录。该目录包含基于 ant-design-vue 的通用业务组件、布局组件、第三方组件封装以及业务专用组件。

5) store 目录

Vuex 状态管理，实现全局状态的定义、模块化管理及数据持久化。该目录采用模块化的状态管理方式，将不同业务模块的状态进行分离管理，包含用户信息、系统配置、业务数据等全局状态。

6) utils 目录

工具函数库，包含加密解密、权限验证、HTTP 请求封装、数据处理等通用功能。该目录提供了项目中常用的工具函数，如日期格式化、数据验证、文件处理等。

(2) public 目录

静态资源根目录，包含应用运行所需的静态文件和配置。其中主要包括：

1) cdn 目录：外部 CDN 资源及配置文件，包含 StarGIS for WebGL 开发包、静态数据文件等。

2) static 目录：系统级静态配置，包含全局配置文件及系统参数。该目录中的 config.js 文件是系统的核心配置文件，定义了与中台、后台及 Server 连接的 API 接口地址、系统参数等关键配置信息。

(3) 根目录配置文件

项目根目录包含 package.json、vue.config.js、babel.config.js 等配置文件，定义了项目的依赖管理、构建配置、编译规则等信息。

1.3.2 核心配置文件说明

config.js 作为系统核心配置文件，位于 public/static/config.js，包含系统名称、登录时长限制以及与中控管理系统、后台服务、Server 连接的 API 接口地址等关键参数信息，在应用启动时进行初始化管理。

1.3.2.1 配置参数说明

(1) PROJECT_NAME：项目名称配置，用于系统左上角标题显示。系统会优先读取中控管理系统中场景组装配置的名称，读取失败时才使用此参数。

(2) TOKENTIME：用户认证 Token 的有效期配置，单位为分钟，系统登录状态下超出该时长将自动登出系统。

(3) RETIME：Token 刷新时间配置，确保用户会话的连续性。

(4) MOUSETIME：鼠标操作超时时间配置，超过该时间一直未移动鼠标将自动登出系统。

(5) VUE_APP_API_BASE_URL：连接中控管理系统，也即 stargis-zk-service 服务地址，用户的登录、场景和功能的使用都依赖于该配置。

(6) VUE_MAP_URL：地图数据服务基础 URL 地址，一般填写 stargis-VRGisServer-service 服务地址，如 “http://127.0.0.1:38083”。功能开发需要用到某个地图数据服务时，若中控管理系统中数据服务 URL 不包括基础 URL，可使用该变量拼接成完整的服务 URL。

(7) VUE_DATA_SERVER_URL：StarGIS TaskServer 服务接口基础地址，用于数据源查询和统计功能，在启动 StarGIS TaskServer 程序的前提下，填写示例如 “http://127.0.0.1:38084”

(8) VUE_DATA_JAVA_URL：Java 后台服务基础 URL 地址，处理地下管线管理和项目档案等相关业务功能时使用。

(9) VUE_SENSORS_URL：星际物联传感平台服务地址，物联网数据接入及管理时使用。

(10) UE4_URL：UE4 引擎服务地址，用于双引擎切换到游戏引擎场景渲染。

(11) lisenceUrl：开发包权限验证，通过：StarGIS Server 提供的校验 URL 对 STARGIS EARTH for WebGLSDK 进行权限验证，验证失败将无法加载三维场景，参数值为：

```
http://<ip>:<port>/stargis/rest/services/System/UtileServer/CheckLocker
```

例如在本地启动 Server 服务的情况下为：

```
http://127.0.0.1:38083/stargis/rest/services/System/UtileServer/CheckLocker
```

(12) VUE_APP_CAS_BASE_URL：单点登录地址，当前版本暂未使用。

(13) VUE_APP_ONLINE_BASE_URL：文件预览路径，当前版本暂未使用。

1.3.2.2 配置加载机制

config.js 文件在 HTML 模板中通过 script 标签引入，确保在 Vue 应用初始化前完成加载。系统采用 window._CONFIG 对象作为全局配置容器，通过该对象暴露所有配置项，实现了配置的全局访问。在实际项目二次开发过程中，可自行增加属性供需使用。

1.3.3 开发规范与命名约定

1.3.3.1 组件结构规范

项目采用标准的 Vue 组件开发规范，确保代码的一致性和可维护性。相关规范主要包括：

(1) 单文件组件必须遵循 `template`、`script`、`style` 的顺序排列，每个部分都有明确的职责分工。`template` 部分负责页面结构定义，`script` 部分包含组件逻辑和数据处理，`style` 部分定义组件样式。组件内部按 `data`、`computed`、`methods`、`created` 等生命周期钩子的顺序组织代码，确保代码的可读性和维护性。样式采用 `scoped` 作用域，避免全局样式污染，确保组件的样式独立性。

(2) 组件内部代码必须按照特定的顺序组织，首先是 `data` 定义，其次是 `computed` 计算属性，然后是 `methods` 方法定义，最后是生命周期钩子。这种组织方式确保了代码的逻辑性和可读性。此外，组件内部的 `name` 属性须与文件名保持一致，便于开发工具的识别和调试。

(3) 项目基于 `ant-design-vue` 组件库进行界面开发，所有 UI 组件须优先使用 `ant-design-vue` 提供的组件，如 `a-button`、`a-table`、`a-form`、`a-modal` 等。

(4) 样式文件采用 `Less` 预处理器，支持变量定义、嵌套规则、混合函数等高级特性。响应式设计必须考虑不同设备和屏幕尺寸的适配，确保用户界面的友好性。组件的样式覆盖须遵循相关设计规范，避免破坏组件的原有样式结构。

1.3.3.2 目录与文件命名规范

(1) 目录规范

`views` 目录下的每个业务模块都应有独立的子目录，如 `views/maps/`、`views/system/`，每个模块下的组件按功能进一步细分。

业务模块功能组件统一放在 `src\views\maps\components` 子目录下，业务模块目录采用语义化命名，组件名应体现其功能或所属模块。如 `PlanningApproval` 文件夹表示规划审批模块、`HeightControl` 文件夹表示控高分析功能，`HomeXXBody.vue` 表示“某模块下所有功能区主体”，`HomeXXFooter.vue` 表示“某模块下底部功能区按钮”，`QueryStatisticPop.vue` 表示“查询统计功能弹窗面板”。

(2) 文件命名约定

Vue 组件文件使用 `PascalCase` 命名，即每个单词首字母大写，单词之间不使用下划线或连字符，如 `MapLegendPop.vue`、`ChartCard.vue` 等。

组件文件名与其在页面中的作用保持一致，如地图相关的弹窗组件命名为 `MapLegendPop.vue`、`LayerControlPop.vue`，而不是简单的 `Pop.vue`，避免歧义。

工具函数文件使用 `camelCase` 命名，如 `auth.js`、`axios.js`、`encryption.js` 等。

样式文件使用 `kebab-case` 命名，如 `common-btn.less`、`common-pop.less` 等。配置文件使用描述性命名，如 `config.js`、`router.config.js` 等。

1.3.3.3 项目管理规范

(1) API 接口规范

API 接口调用优先通过 api 目录下的封装模块进行，避免直接使用 axios 进行接口调用。接口定义须包含完整的参数说明和返回值类型，确保接口调用的正确性和可维护性。错误处理必须统一，包含网络错误、业务错误、权限错误等多种错误类型的处理机制。

(2) 状态管理规范

全局状态通过 Vuex 进行管理，避免使用全局变量或事件总线进行状态传递。状态模块须按业务领域进行划分，确保状态管理的清晰性和可维护性。